

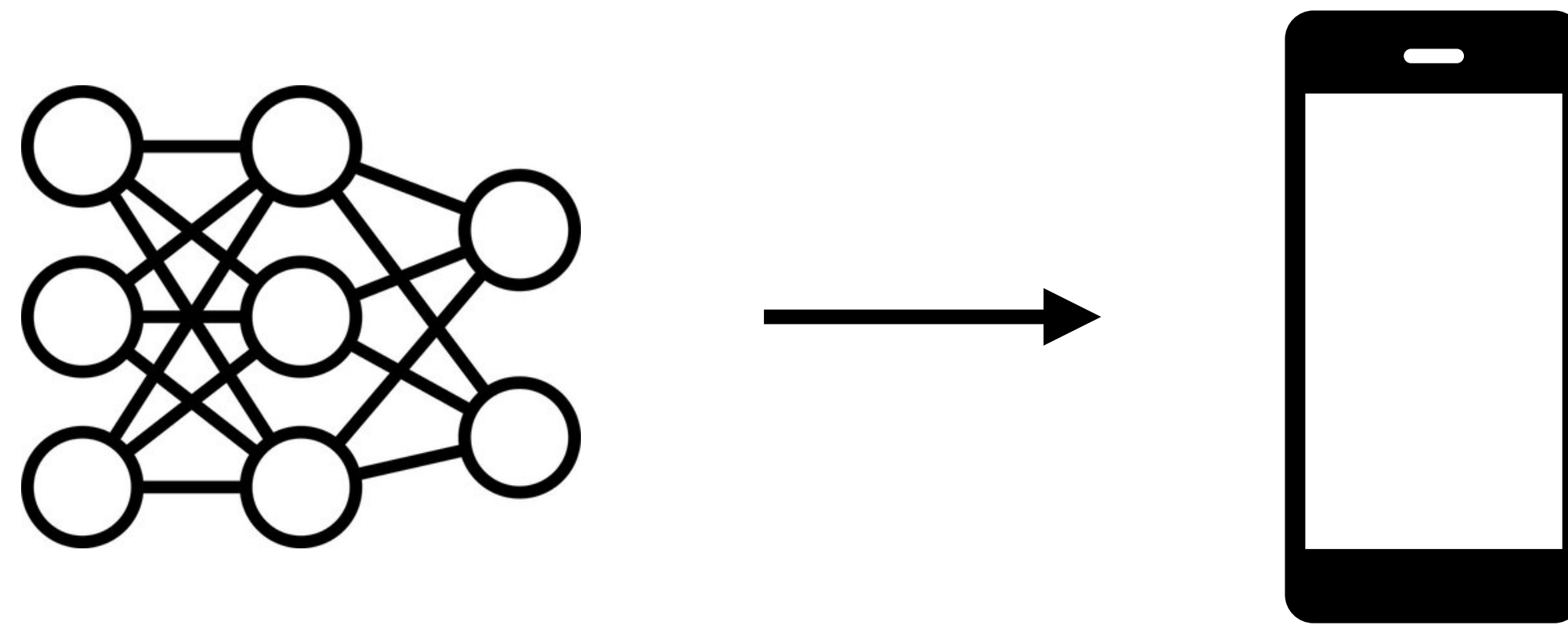
Model size reduction

Agenda

- Distillation
- Quantization

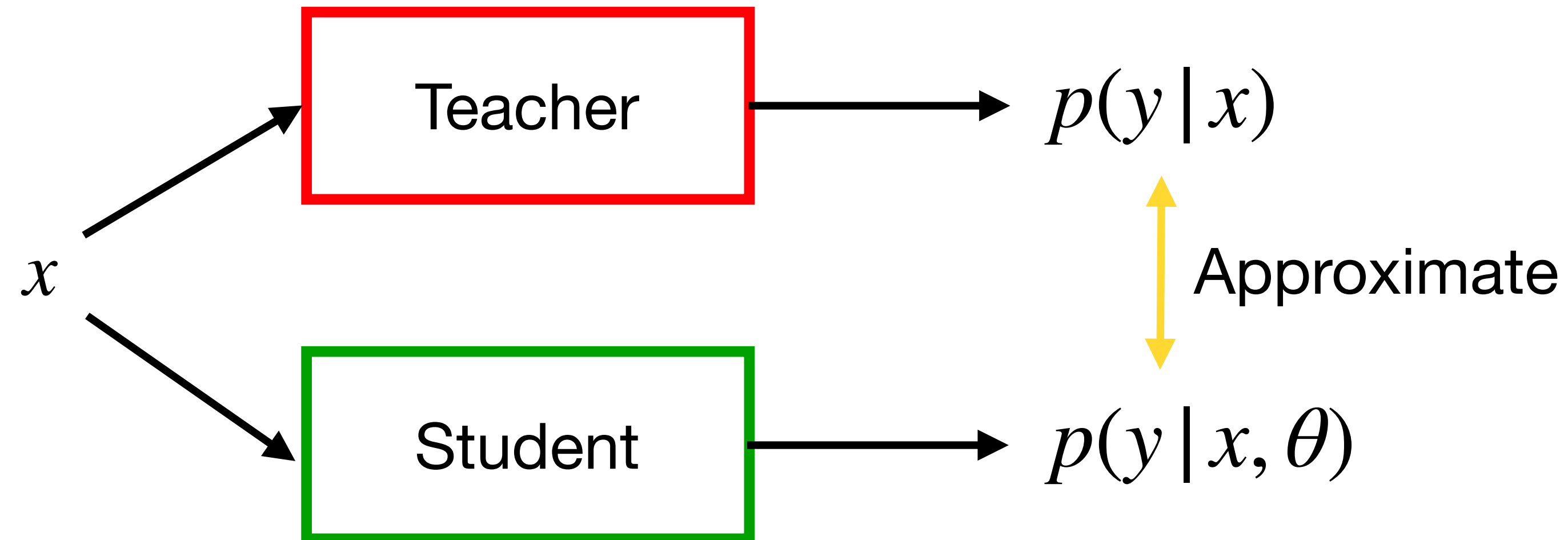
Motivation

- Language models are getting larger every year.
- However, there's often a desire to use models locally on portable devices.
- To achieve this, we need to learn how to reduce their size without losing quality.



Distillation

- **Knowledge distillation** is the process of transferring knowledge from a large trained model (*teacher*) to a smaller model (*student*) with minimal loss of quality.
- Distillation is achieved by bringing the student's outputs closer to the teacher's.



How to approximate?

To approximate probabilities, one of two errors is minimized:

- **KL divergence**

$$KL(p^s \| p^t) = \sum_{k=1}^K p_k^s \log \frac{p_k^s}{p_k^t}$$

- **Cross-entropy**

$$CE(p^t, p^s) = - \sum_{k=1}^K p_k^t \log p_k^s$$

How to approximate?

To approximate probabilities, one of two errors is minimized:

- **KL divergence**

$$KL(p^s \| p^t) = \sum_{k=1}^K p_k^s \log \frac{p_k^s}{p_k^t}$$

- **Cross-entropy**

$$\begin{aligned} CE(p^t, p^s) &= - \sum_{k=1}^K p_k^t \log p_k^s \propto \\ &\propto \sum_{k=1}^K p_k^t \log p_k^t - \sum_{k=1}^K p_k^t \log p_k^s = KL(p^t \| p^s) \end{aligned}$$

Probability smoothing

- Neural networks are often overconfident in their predictions.
- The distribution of p^t turns out to be almost degenerate.
- Approximating the outputs to such a distribution is no better than conventional supervised learning.

Distillation uses smoothed probabilities.

$$\begin{aligned} p_{\tau}^t &= \text{softmax}(l^t, T = \tau) \\ p_{\tau}^s &= \text{softmax}(l^s, T = \tau) \end{aligned} \quad \tau > 1$$

Such labels are called **soft**.

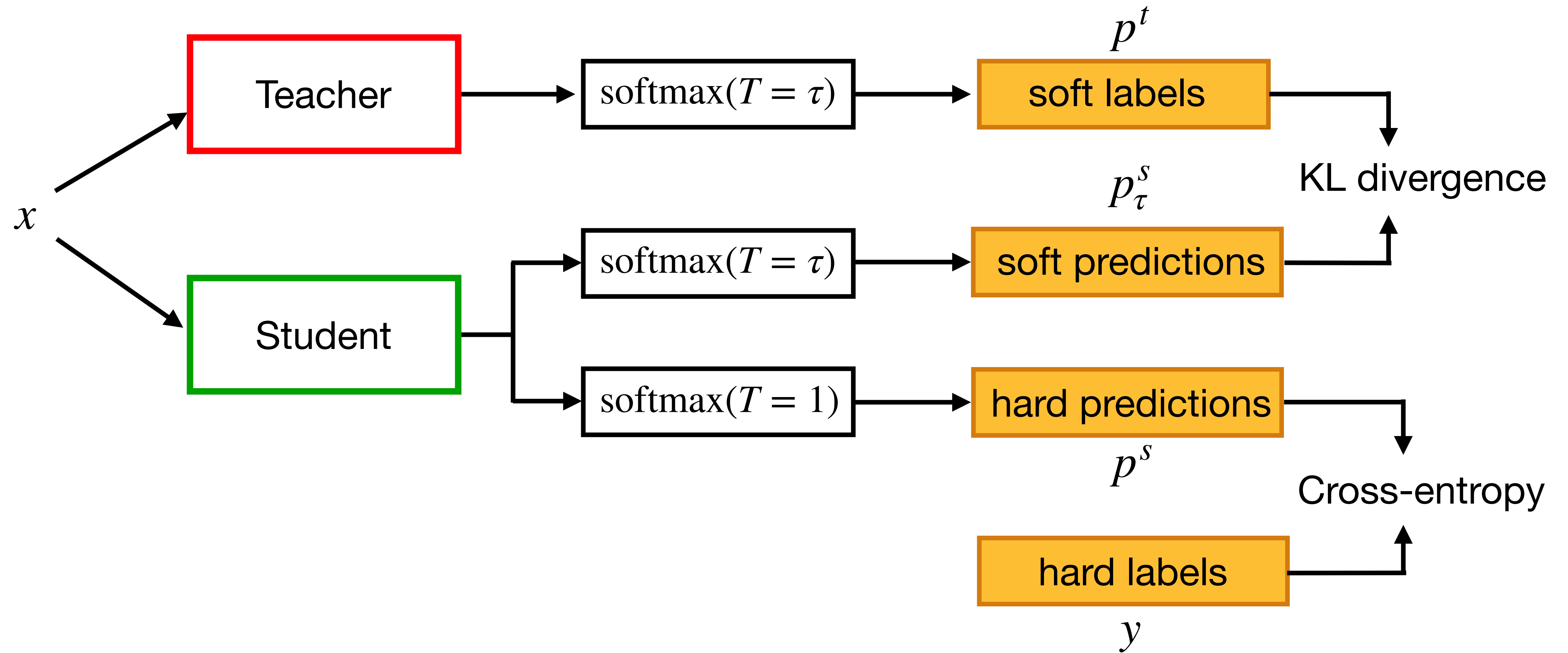
Hard labels

Since we have access to the correct answers, we also add a standard loss.

$$CE(y, p^s) = - \sum_{k=1}^K [y = k] \log p_k^s$$

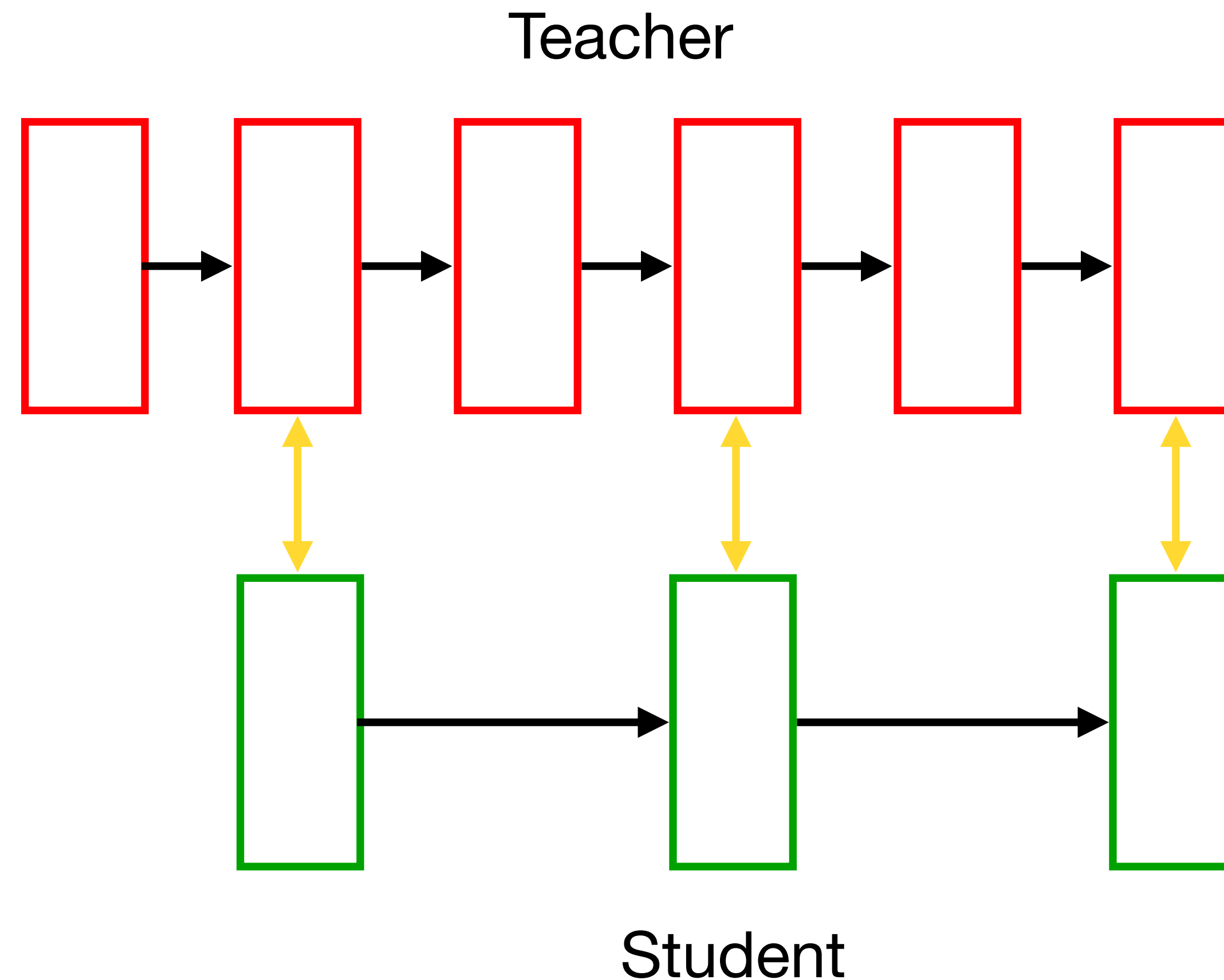
Correct answers y are called **hard** labels.

Training scheme



Approximation of intermediate layers

- We're only approximate the model outputs, but there are also layer outputs!
- We'll try to make the student layer outputs match the teacher outputs.



Approximation of attention layers

The attention matrices have the same size: $[H, \text{seq_len}, \text{seq_len}]$

Let's introduce a loss to minimize the distance between them

$$L_{attn} = \frac{1}{H} \sum_{h=1}^H \|A_h^s - A_h^t\|_2^2$$

Approximation layer outputs

The output layer size can vary between models, so we'll add a projection matrix $W \in \mathbb{R}^{[d_s \times d_t]}$.

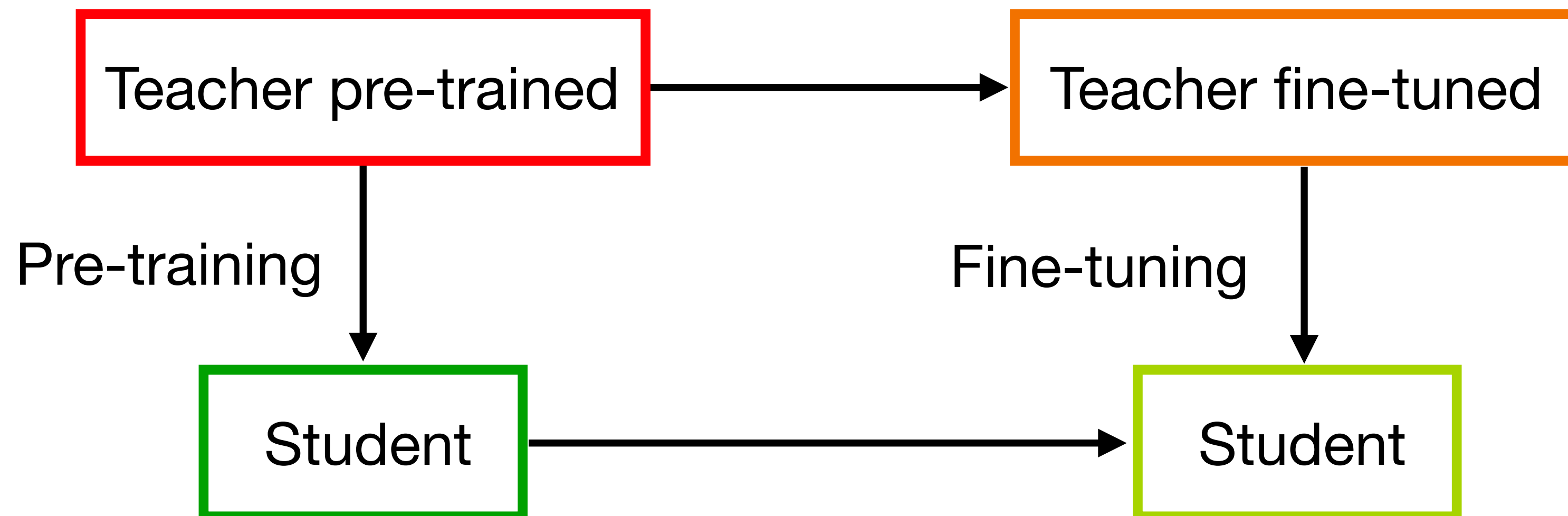
$$L_{hidn} = \|H^s W - H^t\|_2^2$$

The matrix W is trained simultaneously with the student's parameters.

Distillation with pre-training

Almost all transformer models are trained in two stages: pre-training and fine-tuning.

In this case, it's better to first train the student on the pre-training task and then on the fine-tuning task.



Distillation limitations

- Requires access to data
- Time-consuming
- High memory consumption due to the need to store two models

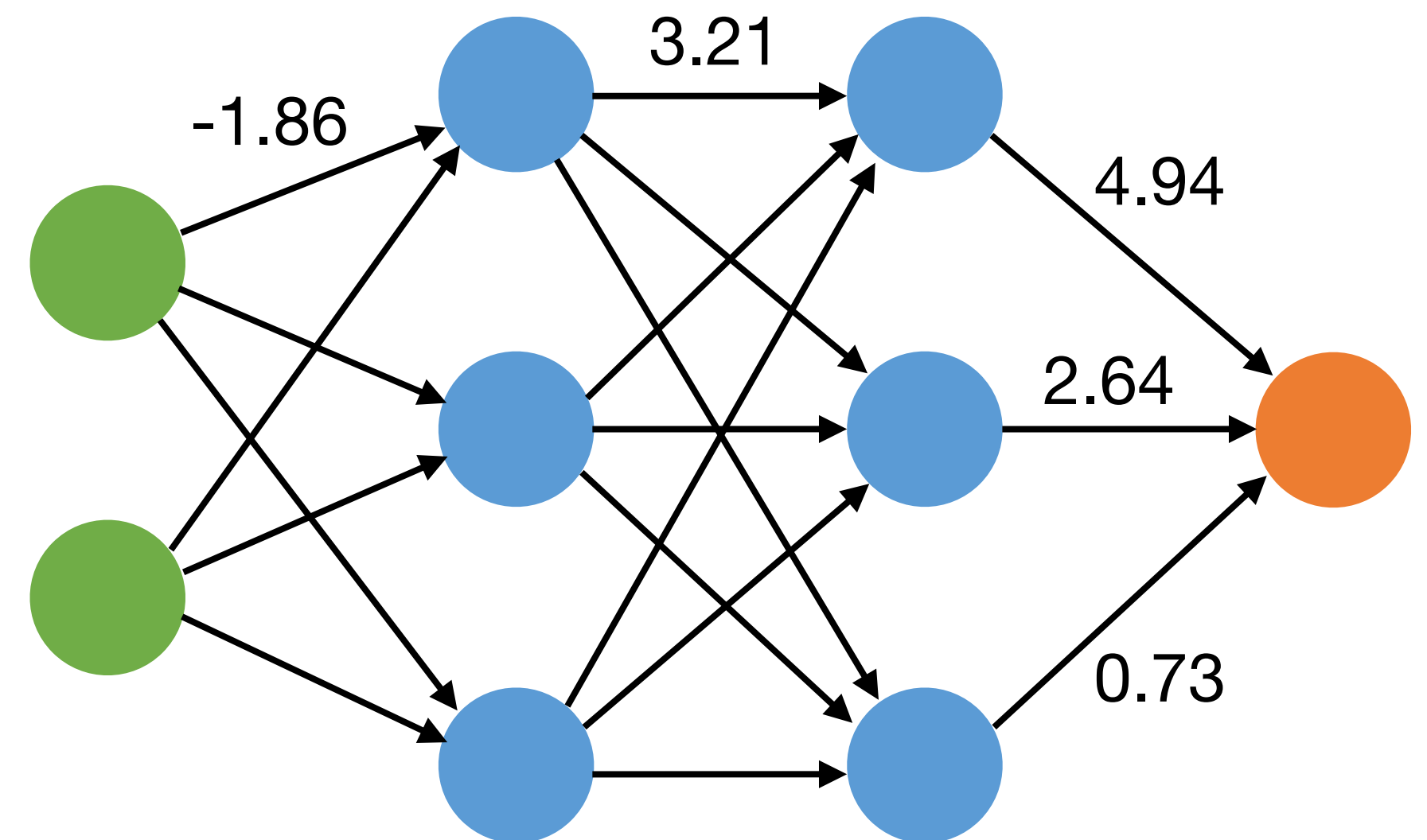
Quantization

Quantization

- By default, neural networks work with numbers in float32 format.
- These are floating point values that occupy 32 bits.
- The more bits used to represent a number, the more it takes
 - time for reading/writing
 - memory for storage
 - time for addition and multiplication operations

Quantization is the reduction of the precision (bit depth) of numbers.

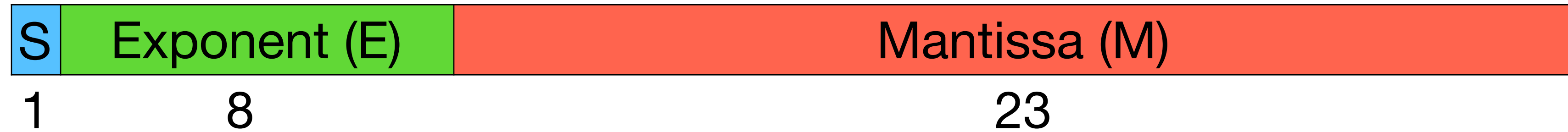
It is most often applied after training.



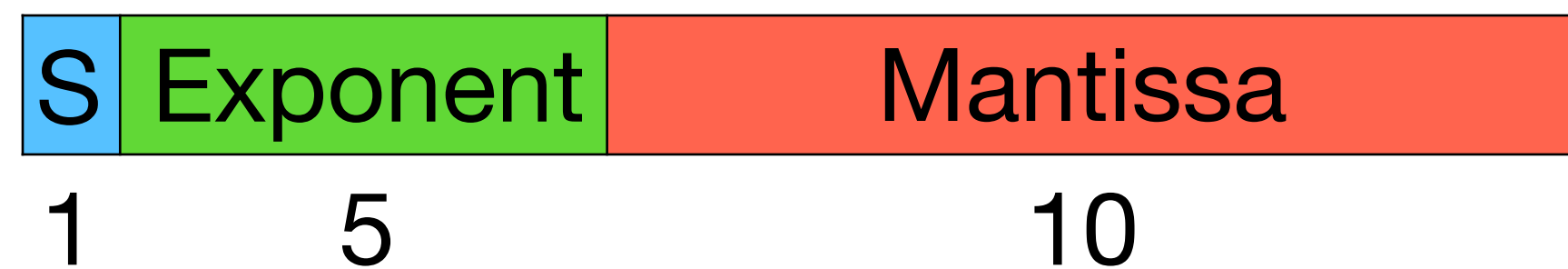
How do computers represent numbers?

float32 (single-precision)

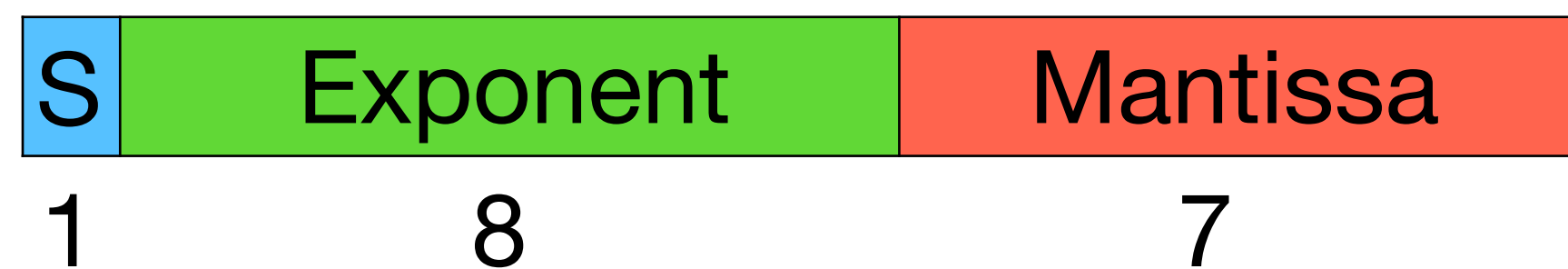
sign



float16 (half-precision)



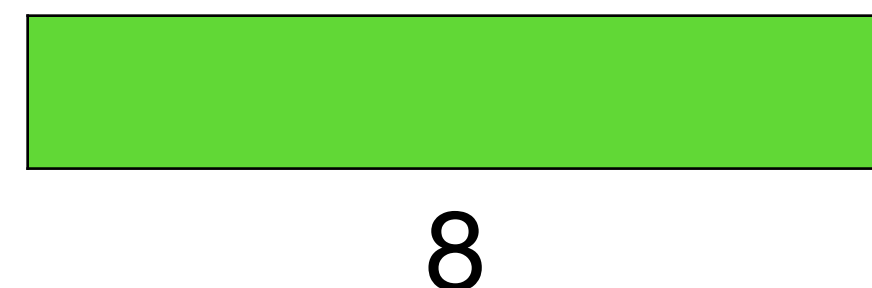
bf16 (brain float)



$$x = (-1)^S \cdot (1, M) \cdot 2^{E-127}$$

To represent large numbers

int8



Why are operations with int8 faster?

- Multiplication in int8 can lead to overflow.
 - Therefore, the result is written to int32. (int8 x int8 -> int32)
 - Then why is multiplication in int8 faster?
1. int8 takes up less memory -> reading and writing are faster.
 2. Many processors support accelerated int8 matrix multiplication.
 3. The conversion to int32 occurs when the result is written.

Quantization

Most often the precision is downgraded from float32 to

- **float16**
- **int8** (float8 is not used because it is almost nowhere defined)

Quantisation in **float16** is done by truncating the mantissa to the required size

Quantisation in **int8** is much trickier

float32 -> int8

Let x be a float32 number and x_q is an int8 quantised x

$$x_q = \text{round}\left(\frac{x}{s} + z\right) \quad x = s \cdot (x_q - z)$$

- s – scale; positive number in float32
- z – zero point; it is stored in int8 and corresponds to 0 in the original float32 number

Such quantisation is called **asymmetric** and is used when $x_q \in [0, 255]$

How to find s and z ?

If we know the range $[a, b]$ of x values and $x_q \in [a_q, b_q]$, then

$$a = s(a_q - z) \quad b = s(b_q - z)$$

Then

$$s = \frac{b - a}{b_q - a_q} \quad z = \frac{ba_q - ab_q}{b - a}$$

How to find s and z ?

If we know the range $[a, b]$ of x values and $x_q \in [a_q, b_q]$, then

$$a = s(a_q - z) \quad b = s(b_q - z)$$

Then

$$s = \frac{b - a}{b_q - a_q} \quad z = \frac{ba_q - ab_q}{b - a}$$

It is important to make sure that 0 is mapped back without errors. That is, that z is an integer.
Therefore

$$z = \text{round} \left(\frac{ba_q - ab_q}{b - a} \right)$$

How to find s and z ?

If we know the range $[a, b]$ of x values and $x_q \in [a_q, b_q]$, then

$$a = s(a_q - z) \quad b = s(b_q - z)$$

Then

$$s = \frac{b - a}{b_q - a_q} \quad z = \frac{ba_q - ab_q}{b - a}$$

It is important to make sure that 0 is mapped back without errors. That is, that z is an integer. Therefore

$$z = \text{round} \left(\frac{ba_q - ab_q}{b - a} \right)$$

If we get $x \notin [a, b]$, we round it to the nearest bound.

$$x_q = \text{clip} \left(\text{round} \left(\frac{x}{s} + z \right), a_q, b_q \right)$$

Symmetric quantization

Used when the range is symmetric, $x \in [-a, a]$.

Quantization maps x to $x_q \in [-127, 127]$.

So we lose one value of the quantised variable, but get rid of the zero point

$$x_q = \text{round}(s \cdot x)$$

How to find a and b ?

We need to quantize the model weights and the input values (activations).

- For weights it is easy - they do not change
- For activations there are three ways:
 - Dynamic quantisation
 - Static quantisation
 - Training time quantisation

How to find a and b ?

We need to quantize the model weights and the input values (activations).

- For weights it is easy - they do not change
- For activations there are three ways:
 - Dynamic quantisation - we calculate bounds on the fly for each activation
 - Static quantisation - we estimate bounds in advance by thresholding n examples.
 - Training time quantisation - use quantisation during training and find bounds from training examples

How to find a and b ?

We need to quantize the model weights and the input values (activations).

- For weights it is easy - they do not change
- For activations there are three ways:
 - Dynamic quantisation - we calculate bounds on the fly for each activation (maximum accuracy, slow speed)
 - Static quantisation - we estimate bounds in advance by thresholding n examples. (lower accuracy, fast speed)
 - Training time quantisation - use quantisation during training and find bounds from training examples (good accuracy, maximum speed, model is tuned for quantisation)

Calibrating boundary values

Min-max: works well for model weights

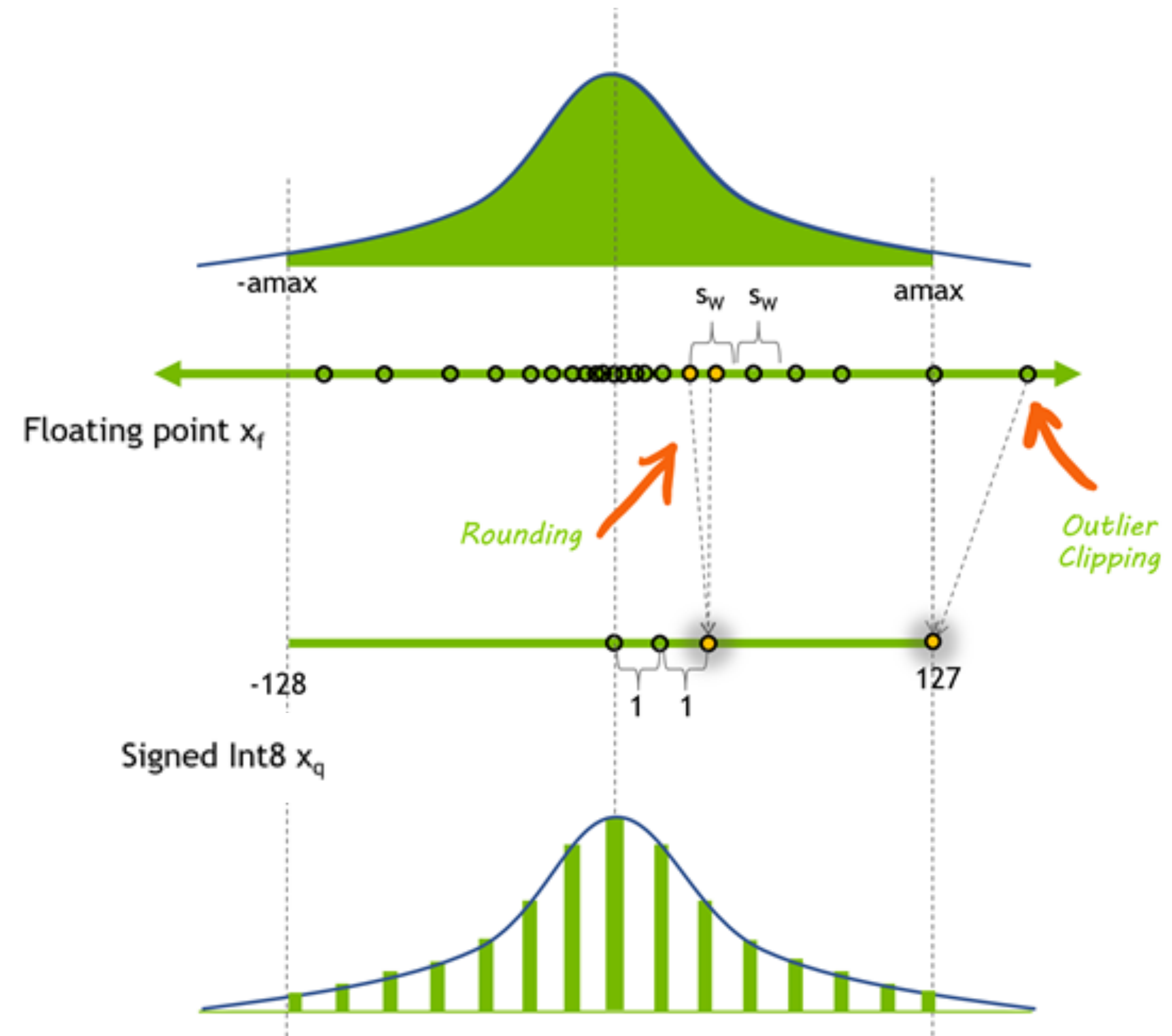
a = minimum found value

b = maximum found value

Moving average min-max: works well for activations

a = moving average minimum value

b = moving average maximum value



Calibrating boundary values

Min-max: works well for model weights

a = minimum found value

b = maximum found value

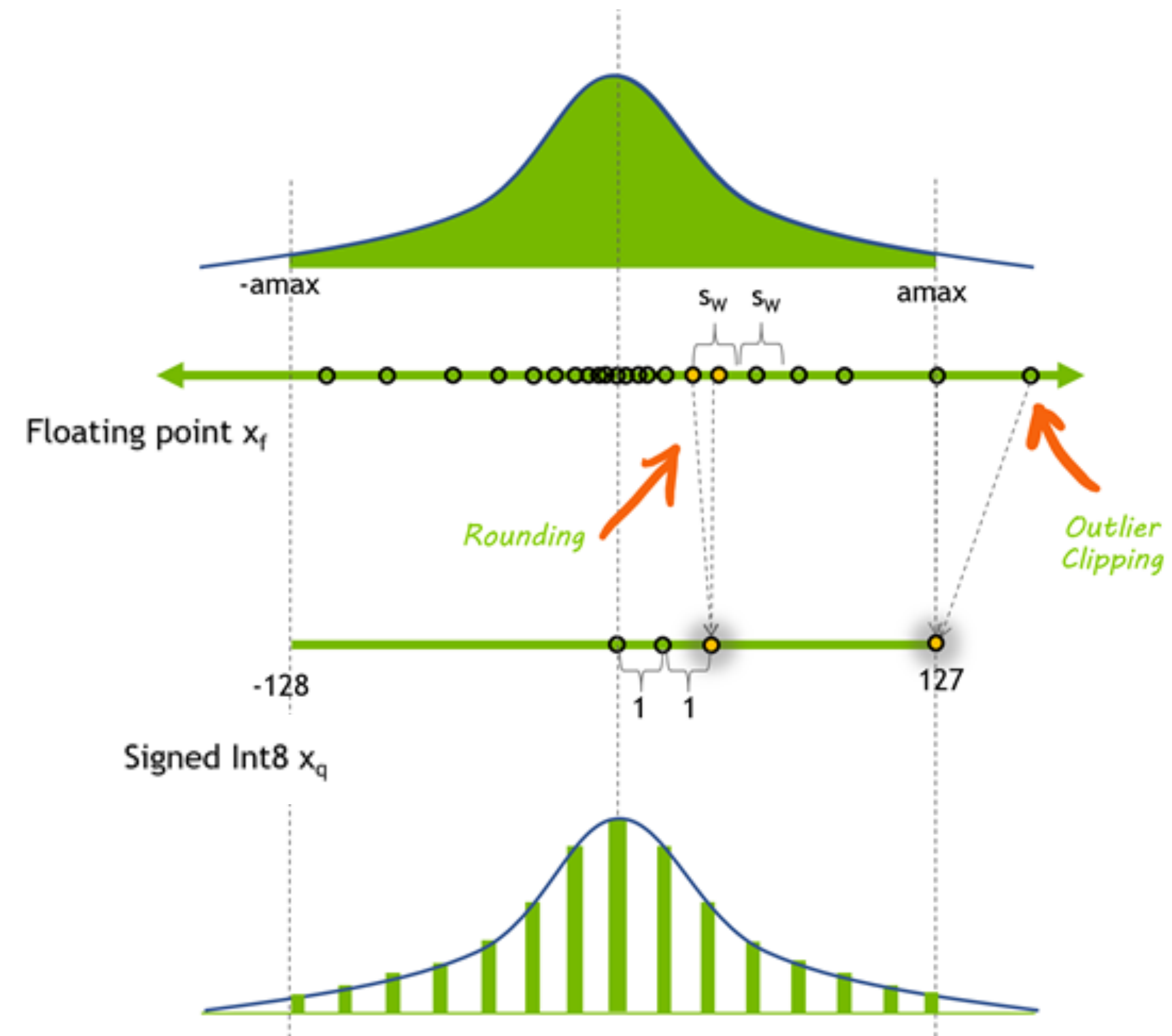
Moving average min-max: works well for activations

a = moving average minimum value

b = moving average maximum value

Histogram-based - form histograms from all activation values, find bounds:

- **Entropy:** minimise KL-divergence between distributions of quantised and original values



Practical tips

- Not all operations need to be quantised.
Matrix multiplications are most important, since they are the most expensive.
Multiplication of two matrices $A \in \mathbb{R}^{[n \times m]}$ and $B \in \mathbb{R}^{[m \times p]}$ requires about $2nmp$ operations.
- For Layer Norm it is better not to apply quantisation, as it may not work stably
- It is best to start with dynamic quantisation.
If the speed is not enough, then move to static quantisation.
At the end, if accuracy is not enough, then to training with quantisation.

Operations with quantised numbers

Addition:

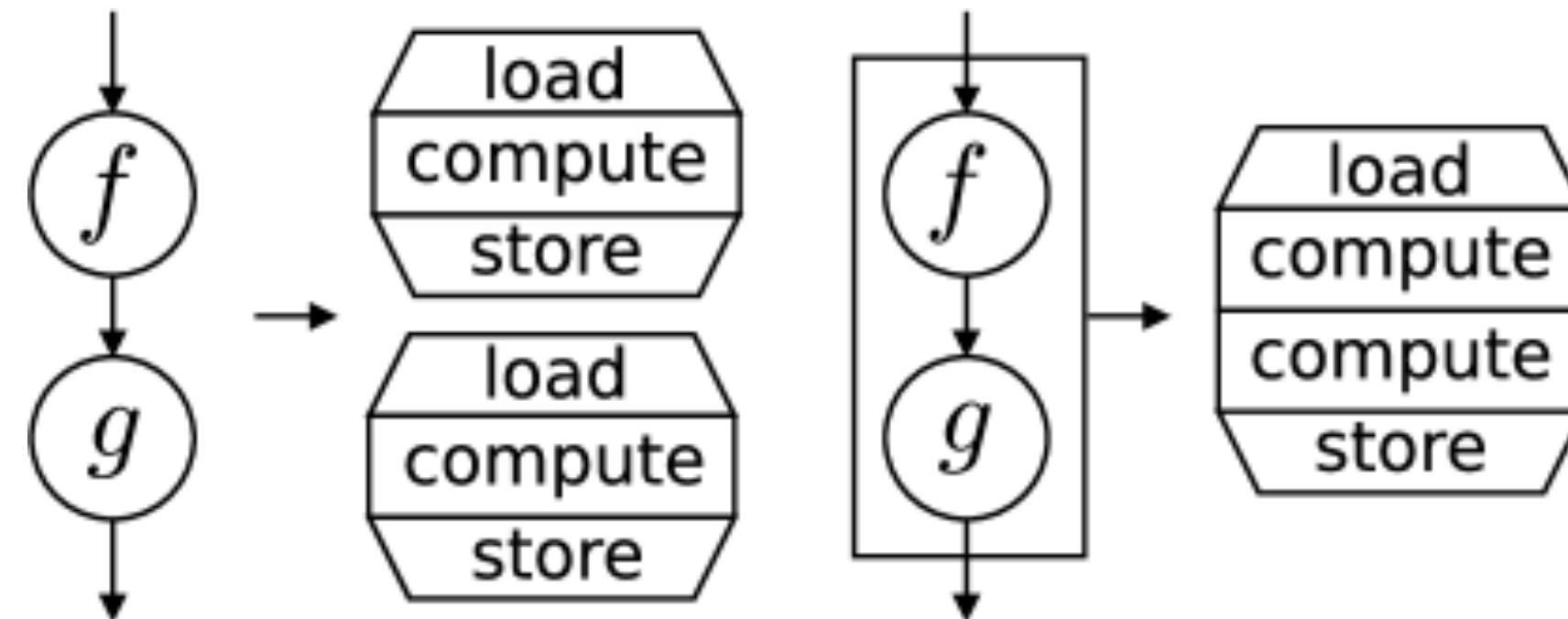
$$y = x_1 + x_2 = s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$
$$y_q = z_y + \frac{1}{s_y} \left(s_1(x_{q1} - z_1) + s_2(x_{q2} - z_2) \right)$$

Multiplication:

$$y = x_1 \cdot x_2 = s_1(x_{q1} - z_1) \cdot s_2(x_{q2} - z_2) = s_y(y_q - z_y)$$
$$y_q = z_y + \frac{s_1 s_2}{s_y} (x_{q1} - z_1)(x_{q2} - z_2)$$

Kernel fusion

- Combine multiple sequential operations into one
- Saves read/write time



Kernel fusion

- Combine multiple sequential operations into one
- Saves read/write time

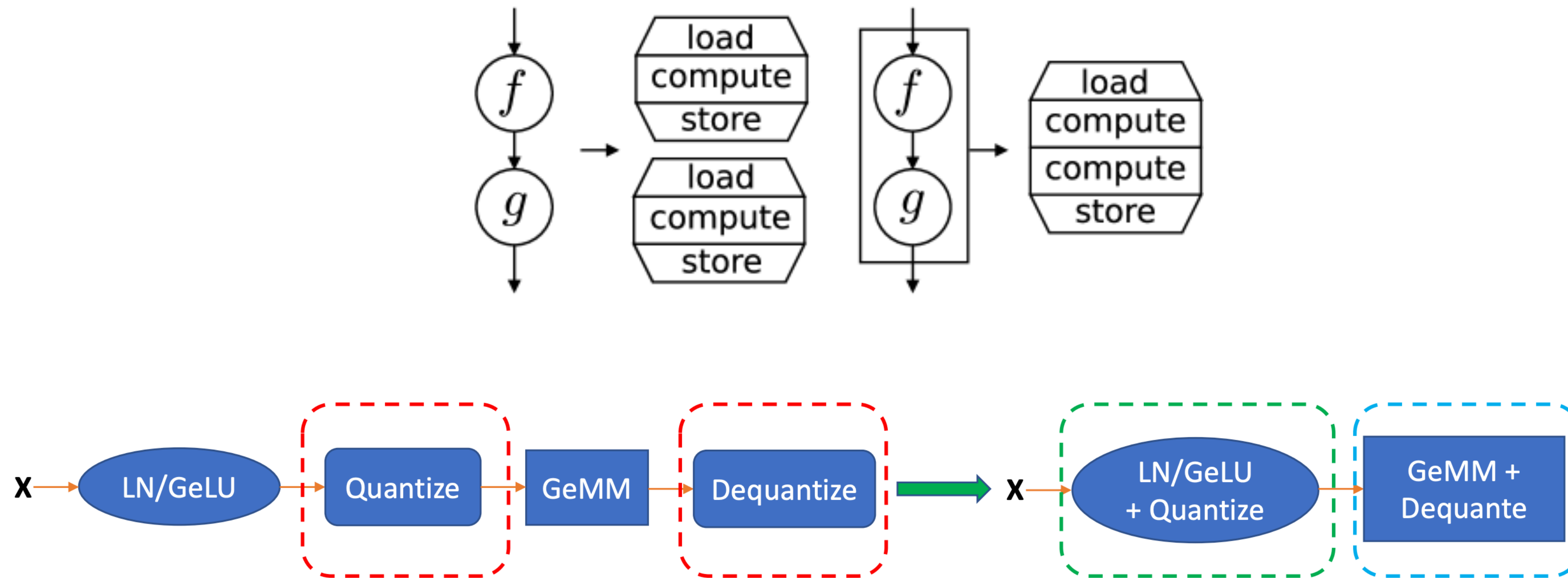
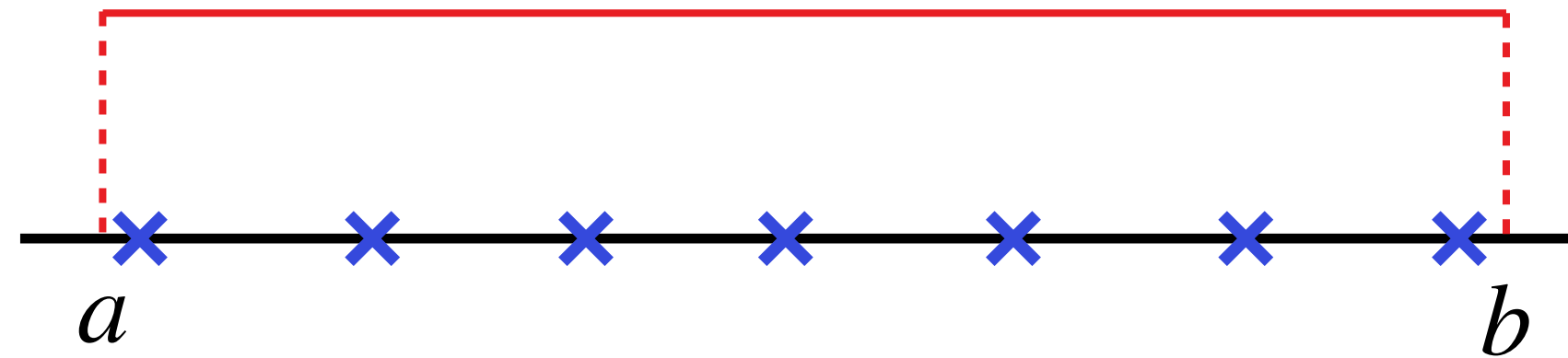


Figure 2: The illustration of normal (left) and our fused (right) INT8 GeMM.

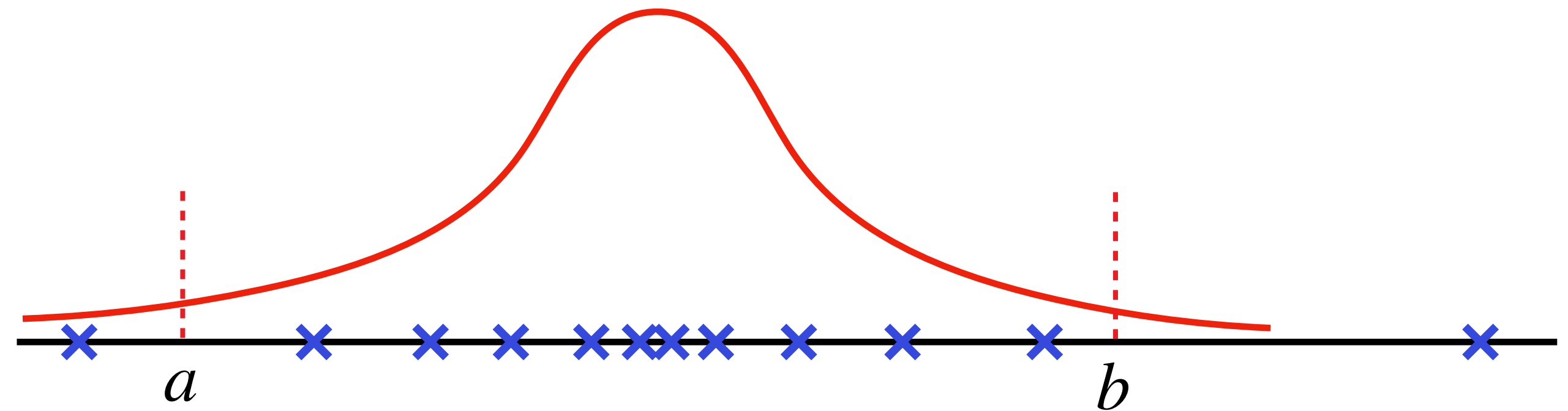
Difficulties of quantization

Values usually are not uniformly distributed

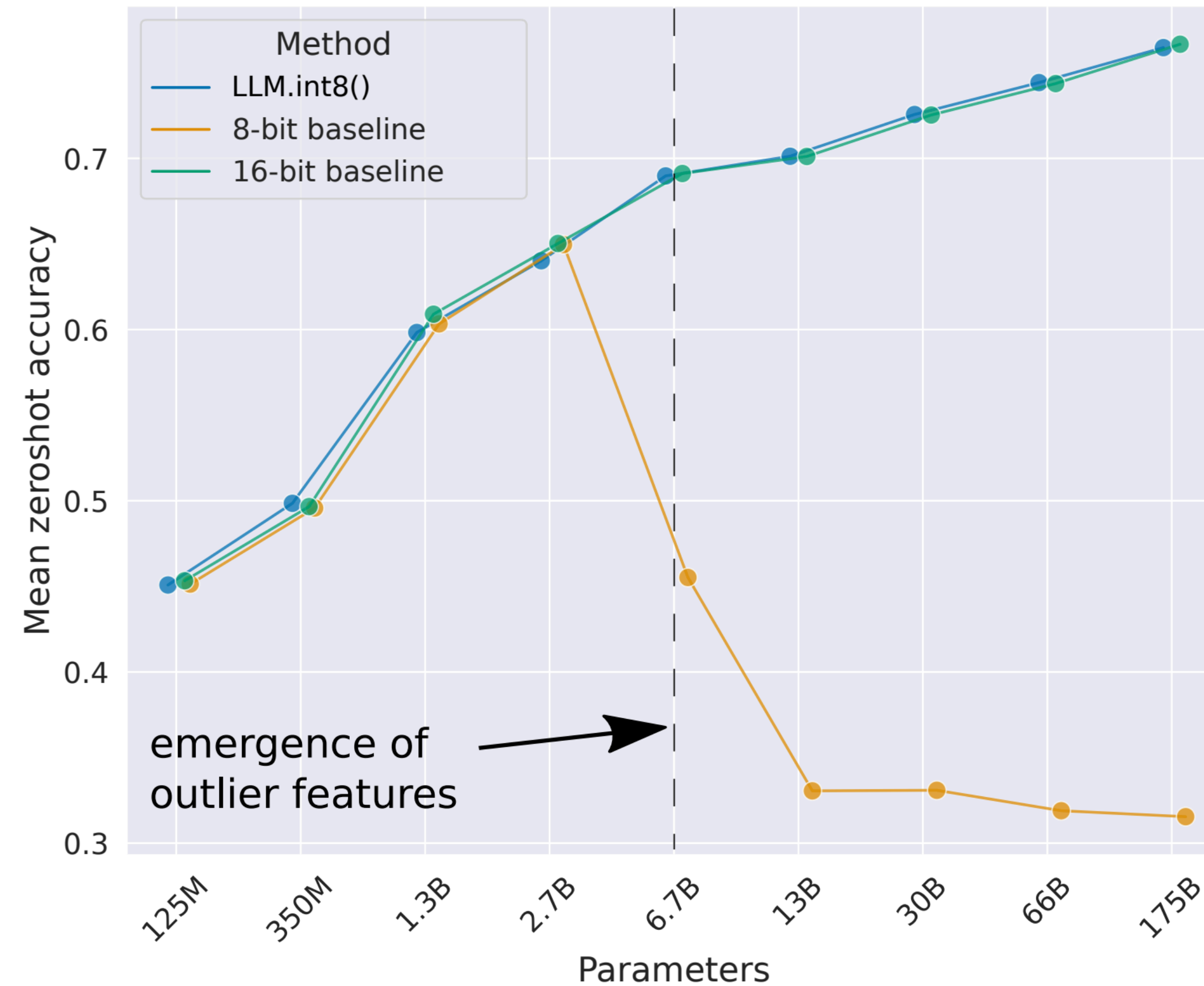
What we want



What we get



Naive quantization works poorly



Naive quantization works poorly

Motivation: The model's activations and weights have quite a lot of outliers, which makes conventional quantization perform poorly.

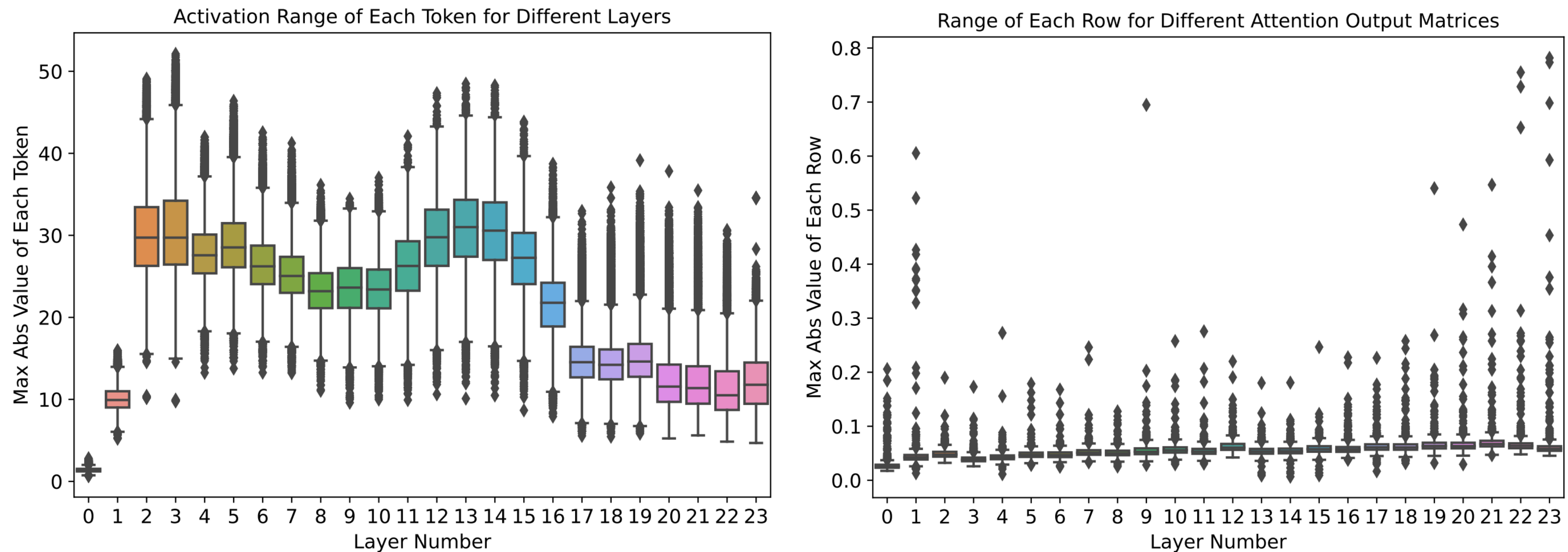


Figure 1: The activation range (left) and row-wise weight range of the attention output matrix (right) different layers on the pretrained GPT-3_{350M}. See Figure C.1 for the results of BERT_{base}.

Naive quantization works poorly

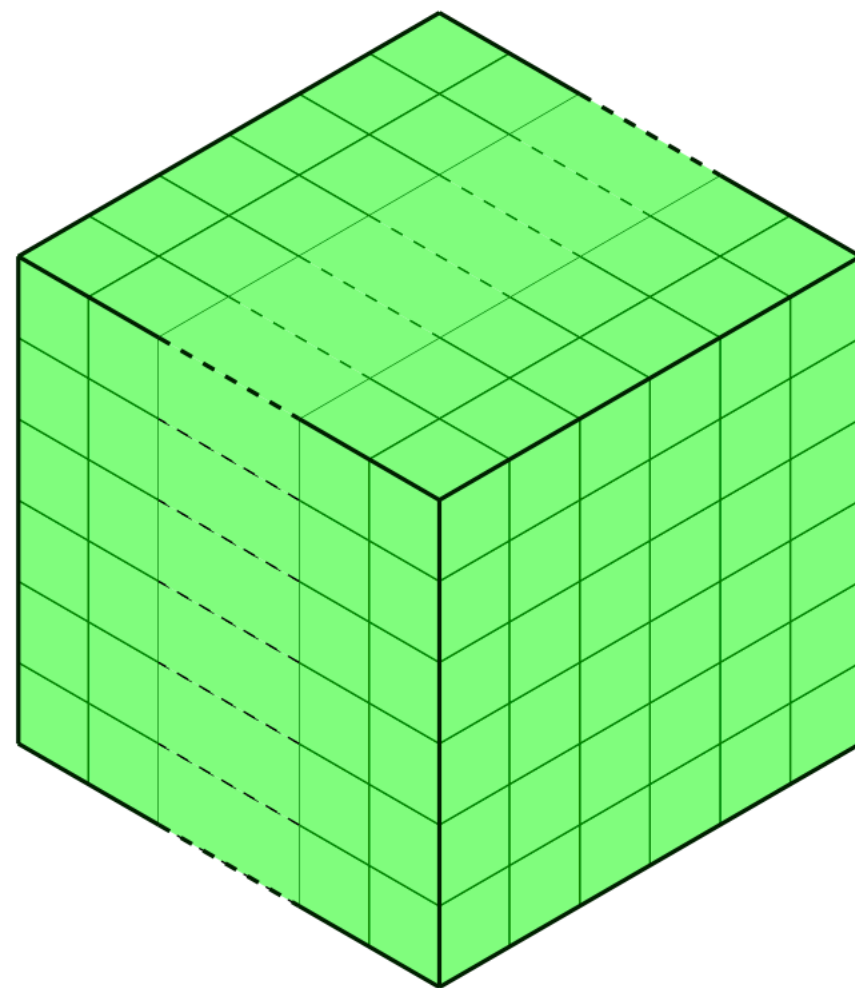
Vanila quantization of both weights and activations significantly degrades quality.

Table 1: Post training quantization results of GPT-3_{350M} on 20 zero-shot evaluation datesets. Here WxAy means x-/y-bit for weight/activation. Particularly, for W4/8, we quantize the MHSA’s weight to INT8 and FFC’s weight to INT4. Please see Table [H.1](#) for the results of all 20 tasks.

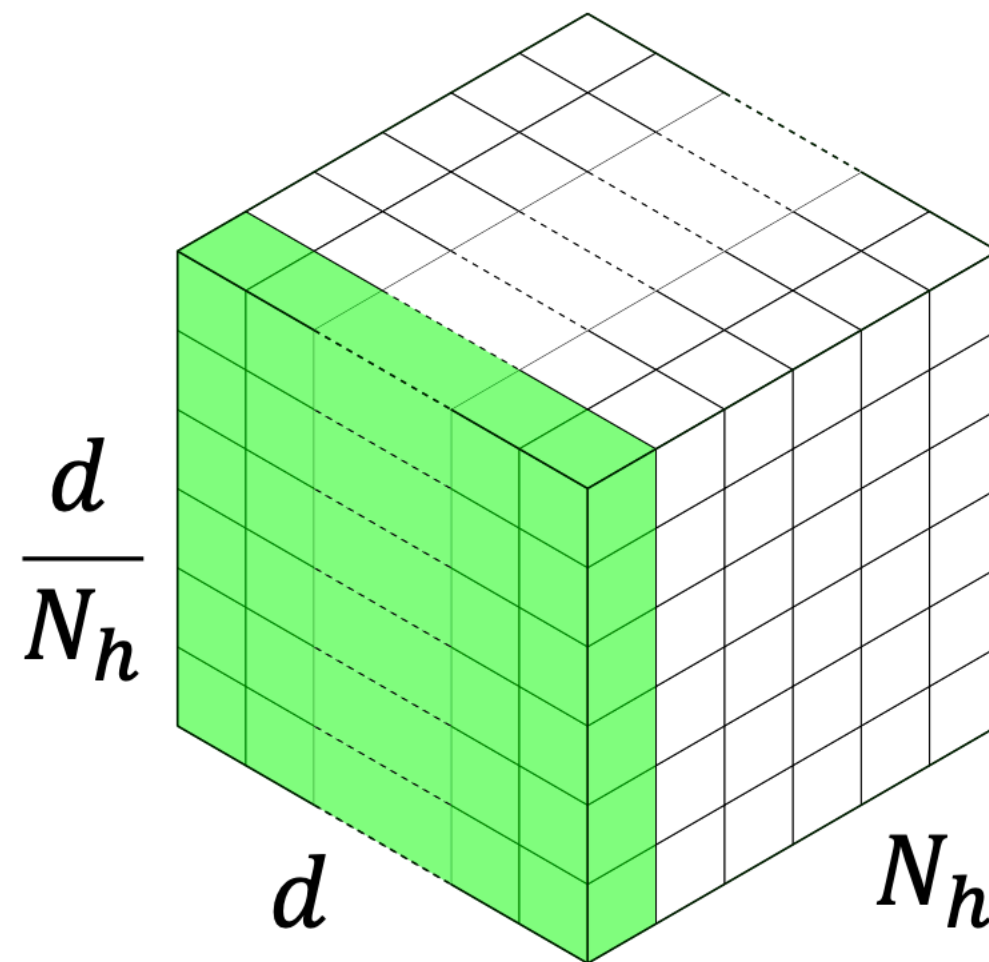
Precision	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)
W16A16	49.3	66.3	29.4	53.8	75.1	38.9	21.5
W8A16	49.3	66.1	29.6	54.2	74.8	38.5	22.1
W16A8	44.7	64.8	28.2	52.7	69.2	37.8	24.6
W8A8	42.6	64.1	28.0	53.1	67.5	37.8	26.2
W4/8A16	0.00	51.4	30.2	52.7	16.1	28.9	1.76e5

ZeroQuant

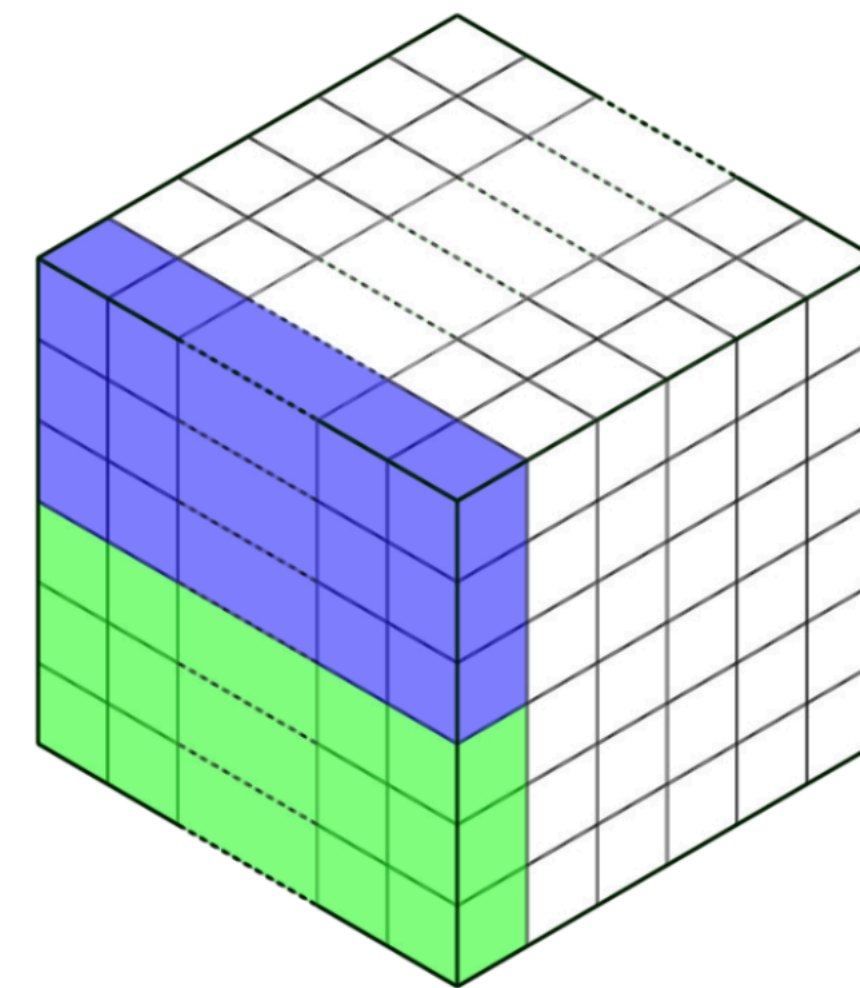
- The first *dynamic* quantization for LLM
- Quantizes tokens (rows of weight matrices) independently of other tokens (rows)



(a) Layer-wise



(b) Group-wise (N_h group)



(c) Group-wise ($2N_h$ group)

ZeroQuant

Significantly improves quality compared to standard static quantization.
2-5x speedup for BERT

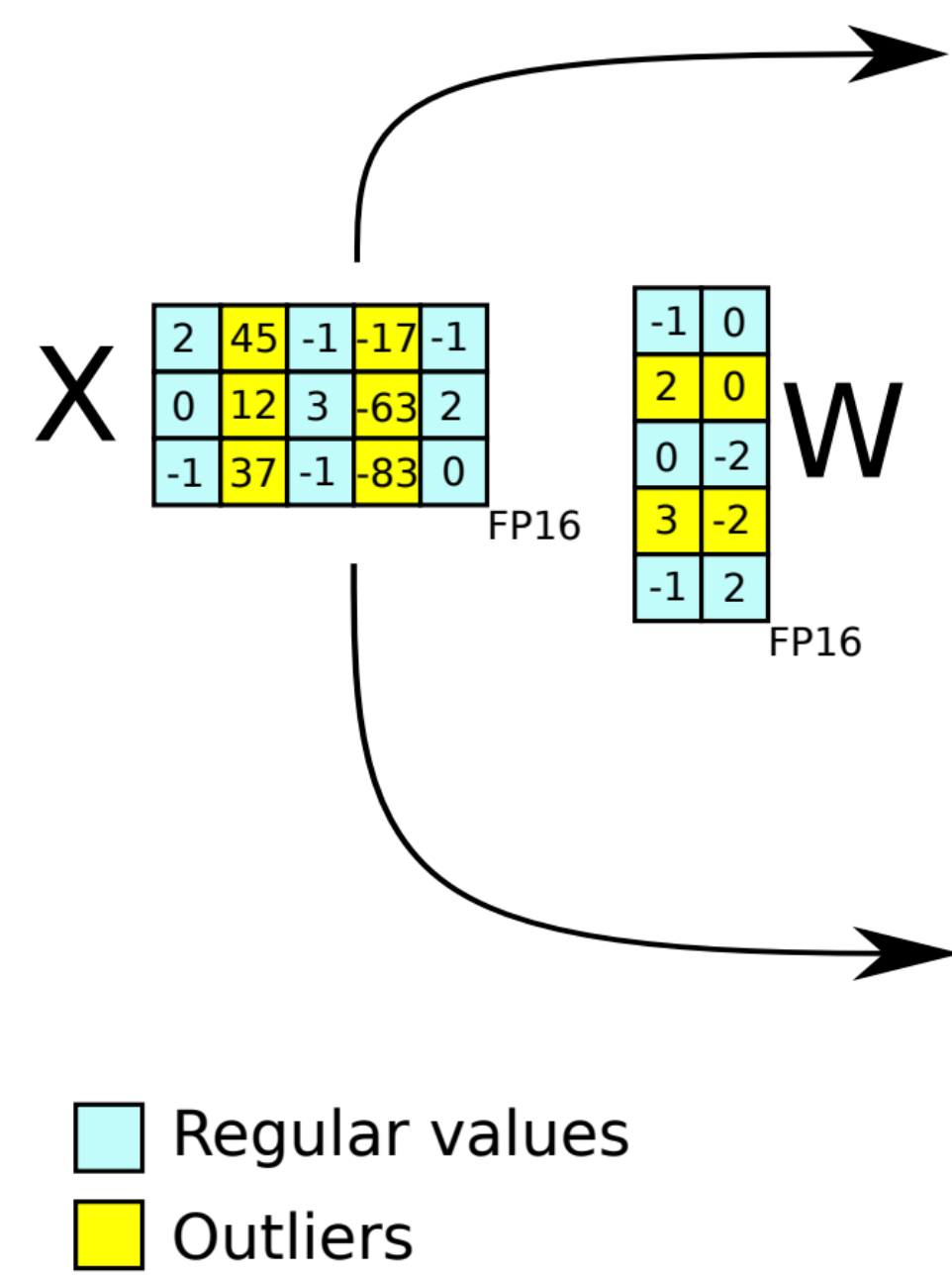
Precision (Method)	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)	Time Cost
W16A16	61.3	71.4	33.6	53.1	82.6	42.4	15.3	N/A
W8A8 (PTQ)	54.8	67.7	16.6	54.5	75.7	40.5	18.9	13 mins
W8A8 (ZeroQuant)	62.6	70.7	33.4	52.7	80.9	42.3	15.7	0
W4/8A16 (PTQ)	0.00	50.4	27.0	50.9	15.8	29.0	1.35e5	13 mins
W4/8A16 (ZeroQuant)	43.9	66.5	30.0	52.7	77.3	39.38	21.9	0
W4/8A16 (ZeroQuant-LKD)	59.4	69.5	31.6	52.7	79.7	41.5	17.6	3 hours
W4/8A8 (ZeroQuant)	46.8	66.4	28.8	52.7	76.2	39.24	24.1	0
W4/8A8 (ZeroQuant-LKD)	48.7	68.1	29.0	52.0	77.4	39.90	18.2	3 hours

Here WxAy means x-/y-bit for weight/activation. Particularly, for W4/8, we quantize the MHSA’s weight to INT8 and FFC’s weight to INT4. LKD is a layer-by-layer knowledge distillation

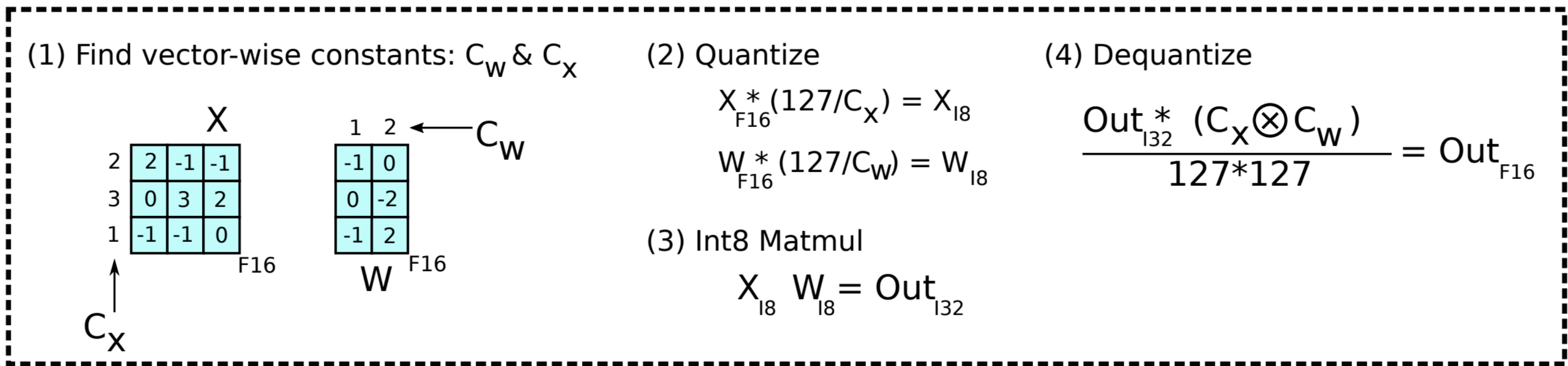
LLM.int8()

Dynamic quantization. Used in Hugging Face (bitsandbytes).

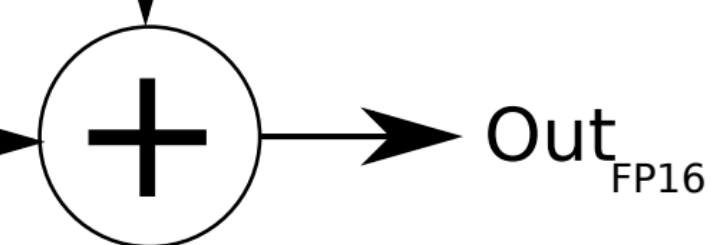
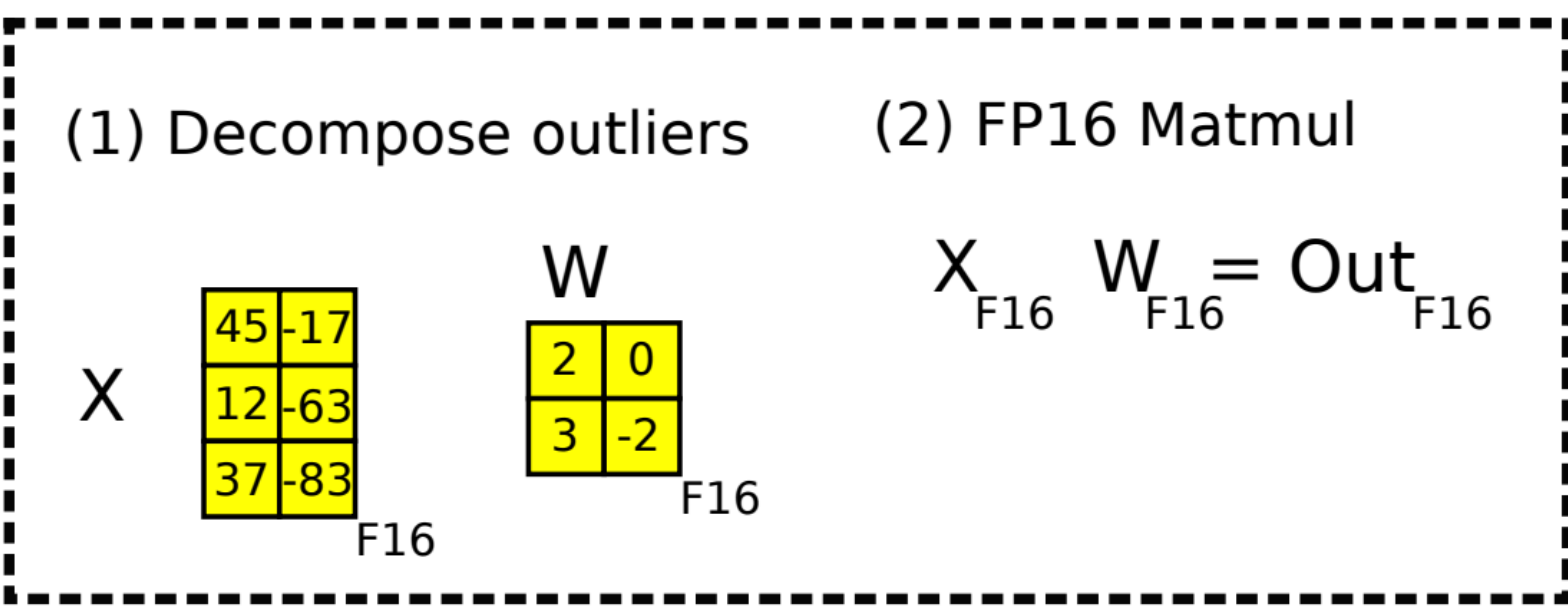
LLM.int8()



8-bit Vector-wise Quantization



16-bit Decomposition



LLM.int8()

Parameters	125M	1.3B	2.7B	6.7B	13B
32-bit Float	25.65	15.91	14.43	13.30	12.45
Int8 absmax	87.76	16.55	15.11	14.59	19.08
Int8 zeropoint	56.66	16.24	14.76	13.49	13.94
Int8 absmax row-wise	30.93	17.08	15.24	14.13	16.49
Int8 absmax vector-wise	35.84	16.82	14.98	14.13	16.48
Int8 zeropoint vector-wise	25.72	15.94	14.36	13.38	13.47
Int8 absmax row-wise + decomposition	30.76	16.19	14.65	13.25	12.46
Absmax LLM.int8() (vector-wise + decomp)	25.83	15.93	14.44	13.24	12.45
Zeropoint LLM.int8() (vector-wise + decomp)	25.69	15.92	14.43	13.24	12.45

Validation perplexity for models of different sizes.

GPTQ

aka Group-wise Precision Tuning Quantization
aka Generative Pre-trained Transformer Quantization

- *Static* quantization. Allows for weight compression to 3 bits.
- The idea is based on alternately quantizing weights with adjustments to unquantized weights.

$$w_q = \operatorname{argmin}_{w_q} \frac{(\operatorname{quant}(w_q) - w_q)^2}{[\mathbf{H}_F^{-1}]_{qq}}, \quad \delta_F = -\frac{w_q - \operatorname{quant}(w_q)}{[\mathbf{H}_F^{-1}]_{qq}} \cdot (\mathbf{H}_F^{-1})_{:,q}$$

- Works fast due to optimizations in Hessian calculation

GPTQ

aka Group-wise Precision Tuning Quantization

aka Generative Pre-trained Transformer Quantization

Method	Bits	OPT-175B				BLOOM-176B			
		Wiki2	PTB	C4	LAMB. $\uparrow$	Wiki2	PTB	C4	LAMB. $\uparrow$
Baseline	16	8.34	12.01	10.13	75.59	8.11	14.59	11.71	67.40
RTN	4	10.54	14.22	11.61	71.34	8.37	15.00	12.04	66.70
GPTQ	4	8.37	12.26	10.28	76.80	8.21	14.75	11.81	67.71
RTN	3	7.3e3	8.0e3	4.6e3	0	571.	107.	598.	0.17
GPTQ	3	8.68	12.68	10.67	76.19	8.64	15.57	12.27	65.10
GPTQ	3/g1024	8.45	12.48	10.47	77.39	8.35	15.01	11.98	67.47
GPTQ	3/g128	8.45	12.37	10.36	76.42	8.26	14.89	11.85	67.86

Table 5: Results summary for OPT-175B and BLOOM-176B. “g1024” and “g128” denote results with groupings of size 1024 and 128, respectively.

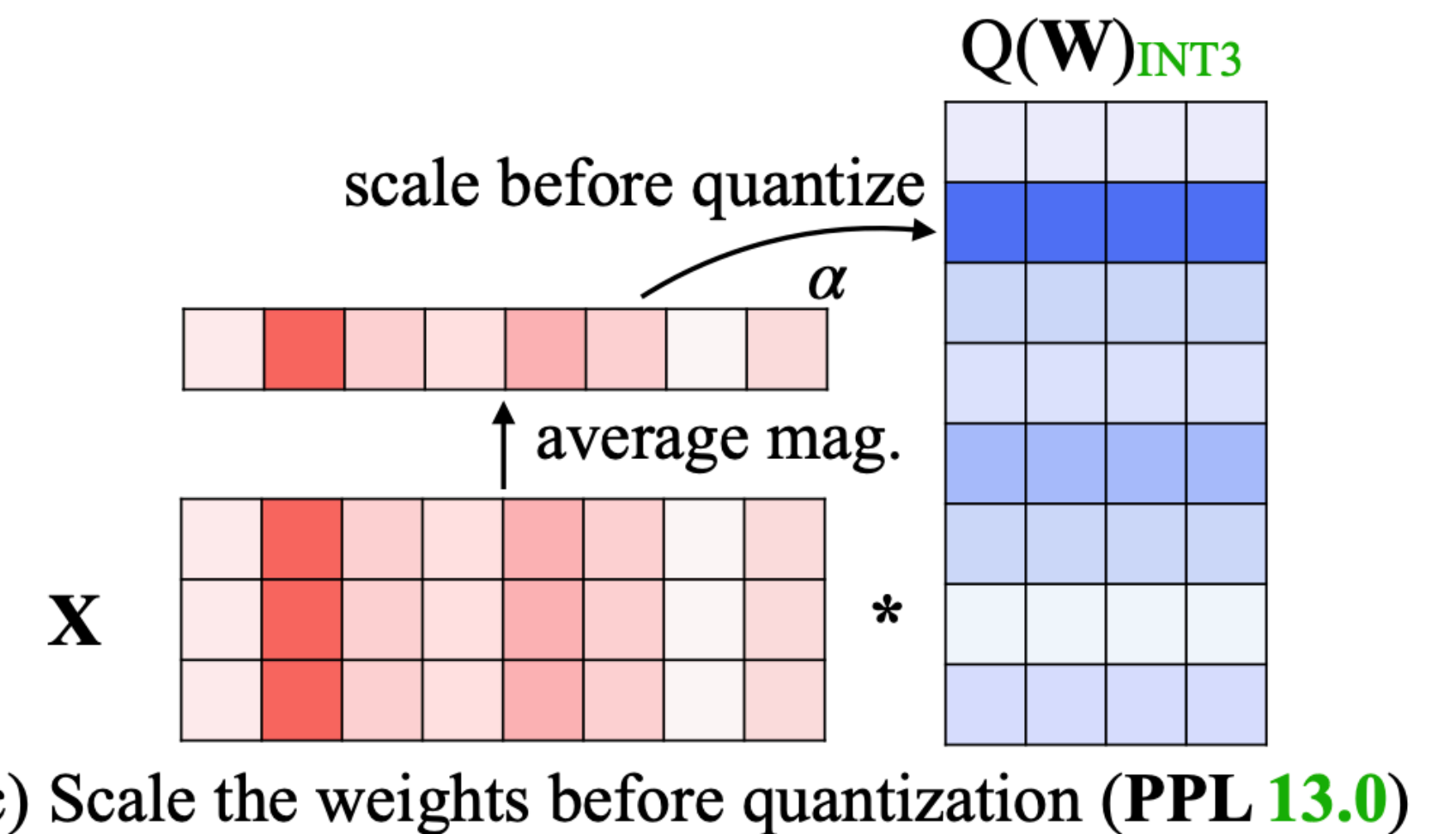
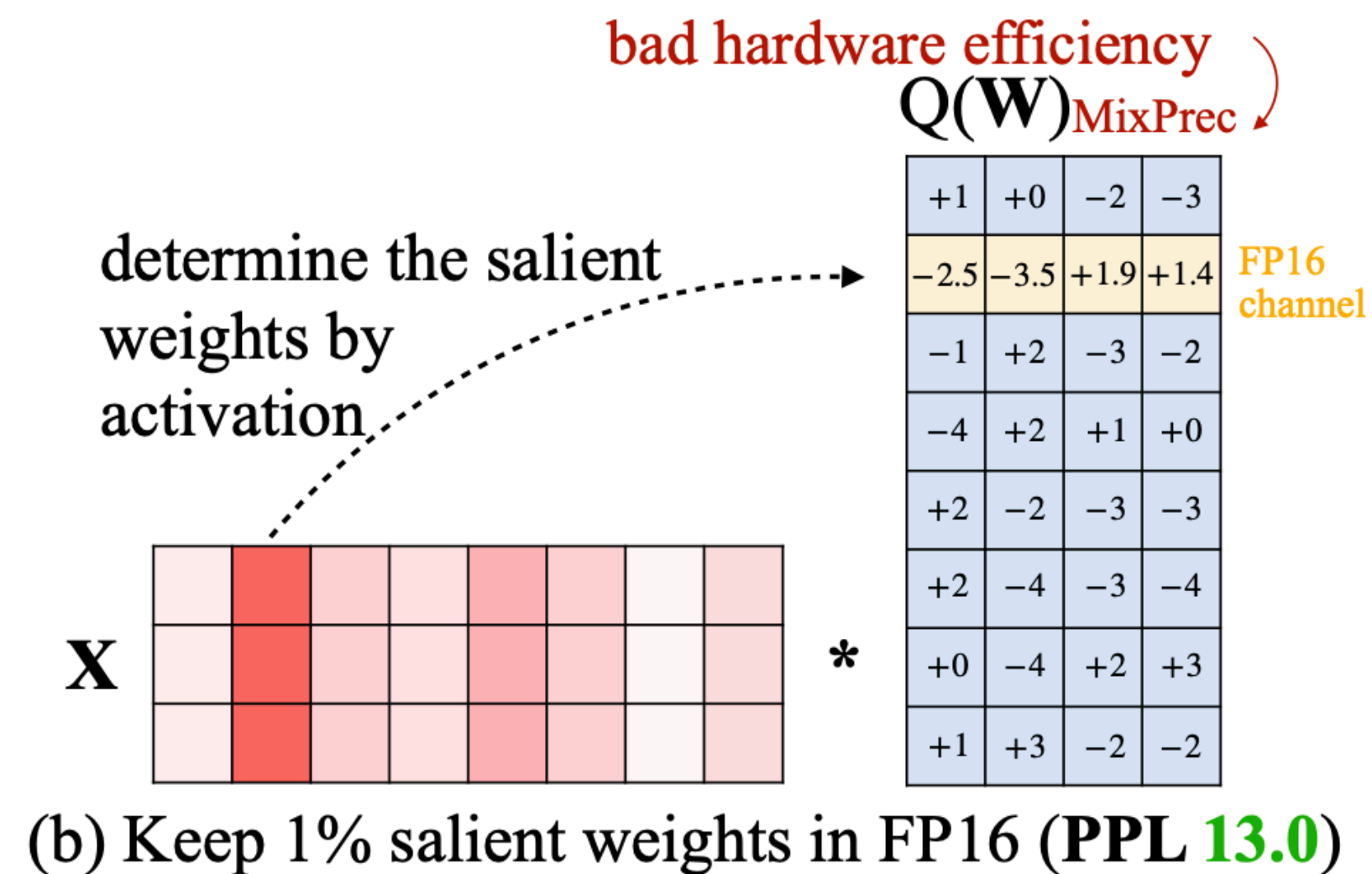
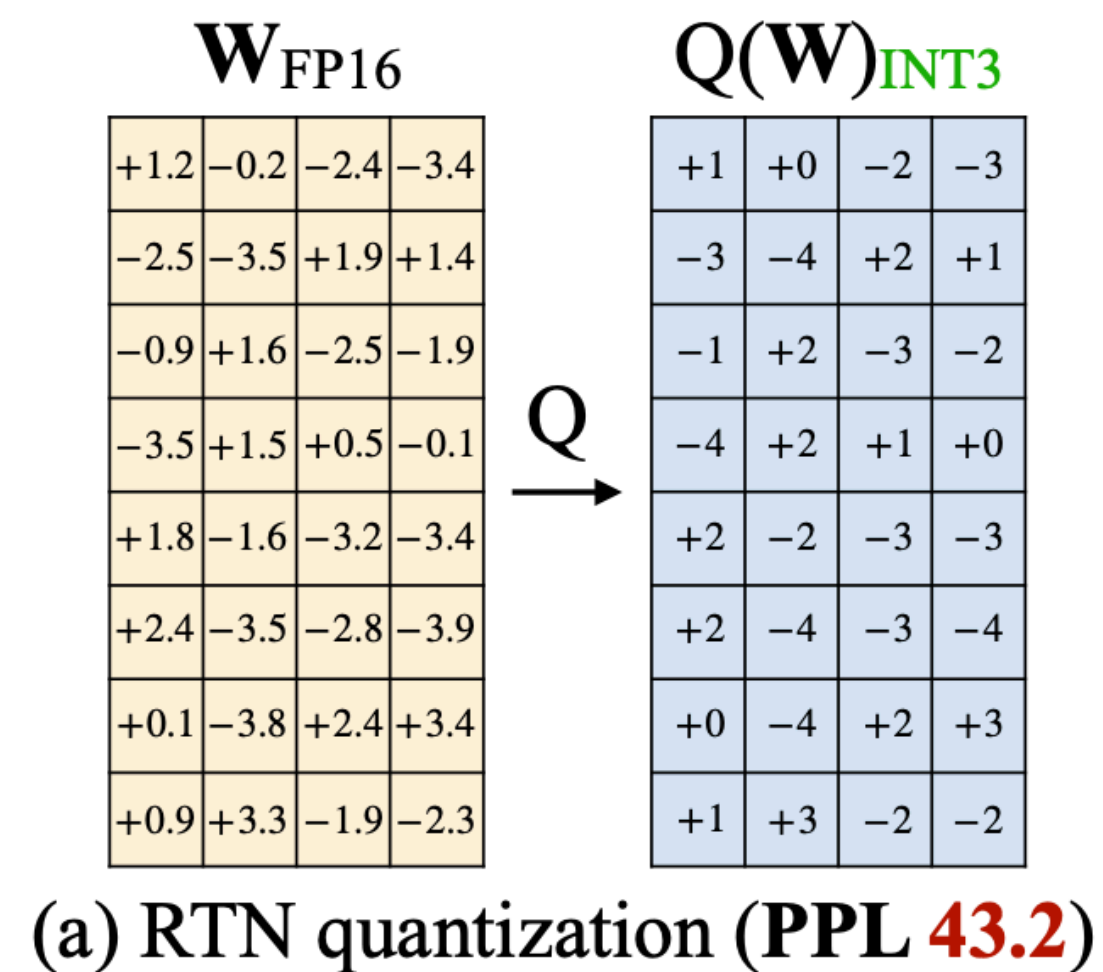
GPU	FP16	3bit	Speedup	GPU reduction
A6000 – 48GB	589ms	130ms	4.53 $\times$	8 $\rightarrow$ 2
A100 – 80GB	230ms	71ms	3.24 $\times$	5 $\rightarrow$ 1

AWQ

Activation-aware Weight Quantization

- *Static* quantization.
- Instead of processing outliers in fp16, AWQ scales them before quantization.

$$Q(\mathbf{w}) = \Delta \cdot \text{Round}\left(\frac{\mathbf{w}}{\Delta}\right) \qquad Q(w \cdot s) \cdot \frac{x}{s} = \Delta' \cdot \text{Round}\left(\frac{ws}{\Delta'}\right) \cdot x \cdot \frac{1}{s}$$

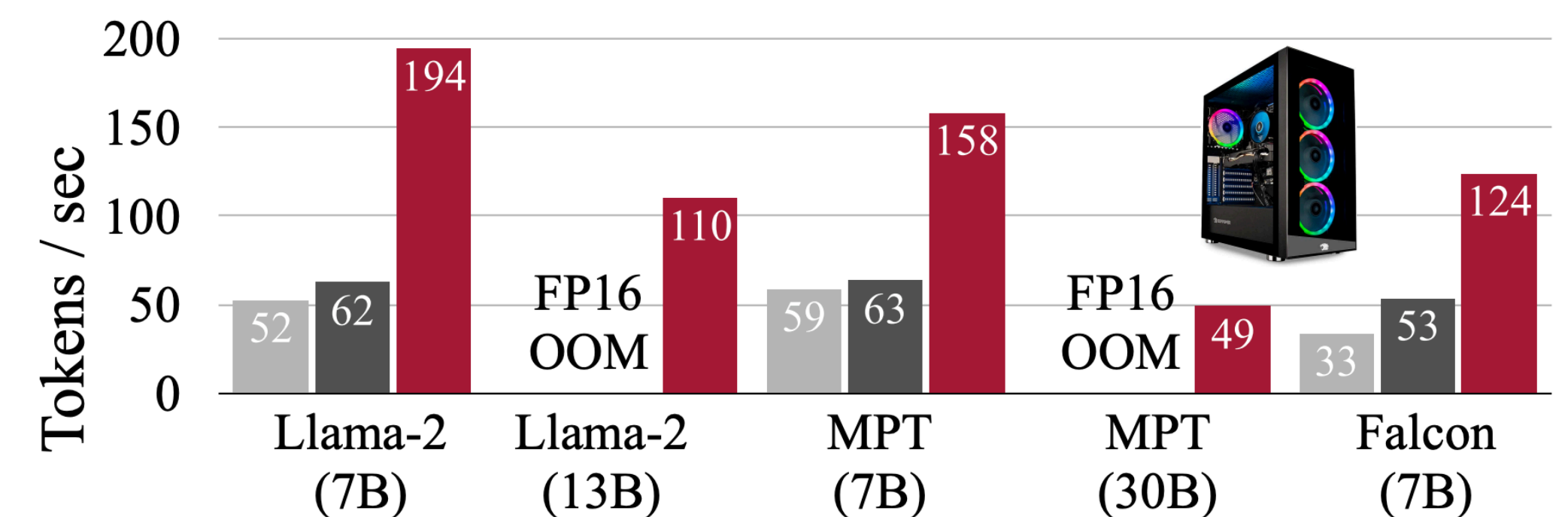


AWQ

Activation-aware Weight Quantization

PPL↓		Llama-2			LLaMA			
		7B	13B	70B	7B	13B	30B	65B
FP16	-	5.47	4.88	3.32	5.68	5.09	4.10	3.53
INT3 g128	RTN	6.66	5.52	3.98	7.01	5.88	4.88	4.24
	GPTQ	6.43	5.48	3.88	8.81	5.66	4.88	4.17
	GPTQ-R	6.42	5.41	3.86	6.53	5.64	4.74	4.21
	AWQ	6.24	5.32	3.74	6.35	5.52	4.61	3.95
INT4 g128	RTN	5.73	4.98	3.46	5.96	5.25	4.23	3.67
	GPTQ	5.69	4.98	3.42	6.22	5.23	4.24	3.66
	GPTQ-R	5.63	4.99	3.43	5.83	5.20	4.22	3.66
	AWQ	5.60	4.97	3.41	5.78	5.19	4.21	3.62

■ Huggingface (FP16) ■ Ours (FP16) ■ Ours (AWQ, W4A16)

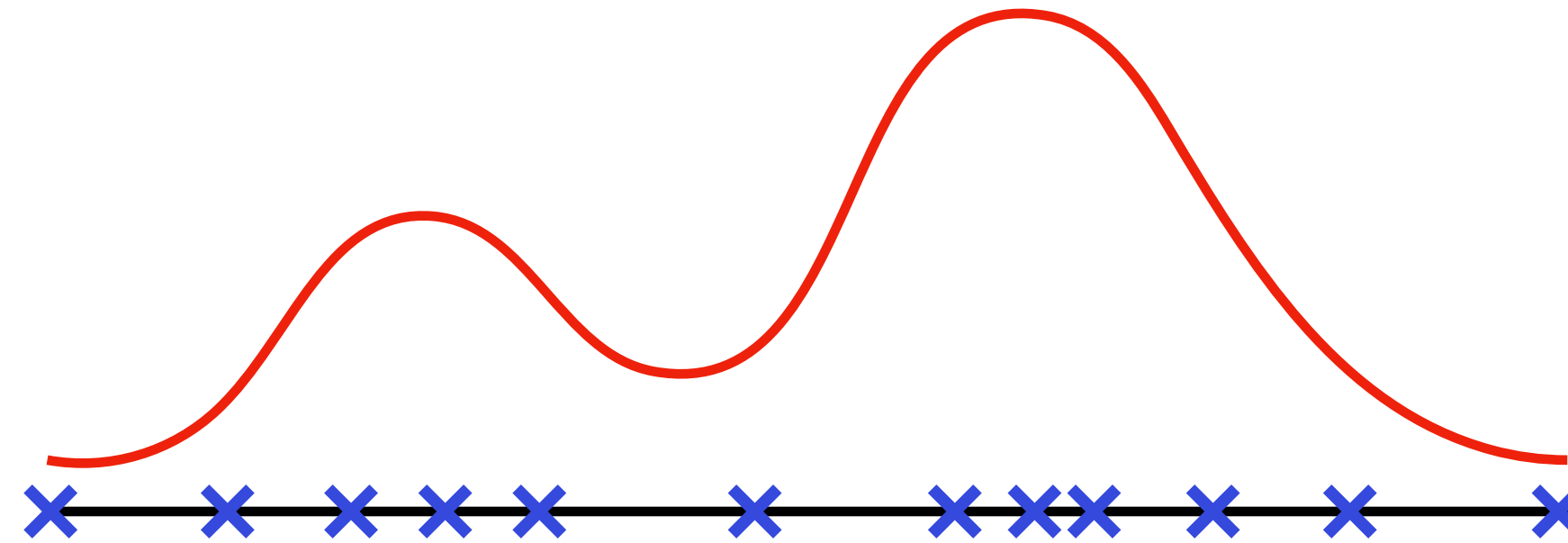


(a) RTX 4090 desktop GPU

- AWQ requires 10 times less calibration data than GPTQ.
- It speeds up inference by almost 4 times.

Nonlinear quantization

Typically, weight values and activations are unevenly distributed. It's useful to increase precision where the density is higher.



There are several ways to do this, but they are used less often due to their complexity.